MINI PROJECT REPORT

AI-BASED PHISHING URL DETECTION SYSTEM

Submitted by

Daniel Raj V
Karthikeyan R
Justin Imre J

Under the guidance of

Ms. Divya Bairavi

Department: _____

College / University: _____

Academic Year: _____

A detailed academic report on machine-learning methods for detecting potentially phishing URLs.

CERTIFICATE

This is to certify that the mini project report entitled **“AI-Based Phishing URL Detection System”** is prepared by **Daniel Raj V, Karthikeyan R, and Justin Imre J** under the guidance of **Ms. Divya Bairavi**. The report presents the problem, proposed workflow, relevant machine-learning methods, and an evaluation plan for identifying potentially phishing URLs.

Guide’s Signature: _____

Head of Department: _____

Date: _____ Place: _____

Institution Seal: _____

DECLARATION

We declare that this report, titled **“AI-Based Phishing URL Detection System,”** has been prepared for academic mini-project purposes by Daniel Raj V, Karthikeyan R, and Justin Imre J under the guidance of Ms. Divya Bairavi.

The report explains the proposed system and summarises the technical ideas presented in the project presentation. Any performance figures copied from the presentation are identified as illustrative and must not be treated as measured results unless confirmed by experiments. Institutional details and implementation-specific information should be completed and verified by the project team before submission.

Student signatures:

1. Daniel Raj V: _____

2. Karthikeyan R: _____

3. Justin Imre J: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, **Ms. Divya Bairavi**, for her guidance and support in preparing this mini project report. Her guidance helped us organise the topic and understand the key stages involved in describing an AI-based phishing URL detection system.

We thank our department and institution for providing an opportunity to explore cybersecurity, web safety, and machine-learning concepts. We also acknowledge the researchers and organisations whose publications and datasets provide background for phishing detection research.

Finally, we thank our classmates, friends, and family members for their encouragement during the preparation of this report.

ABSTRACT

Phishing is a social-engineering attack in which a malicious actor directs a user to a deceptive website in order to collect confidential information such as passwords, one-time passwords, payment details, and personal data. Traditional blacklist-based systems are useful for identifying known malicious URLs, but they may not recognise newly created or modified links immediately. This project describes an AI-based approach that extracts characteristics from URLs and applies machine-learning classifiers to predict whether a URL is likely to be phishing or legitimate.

The proposed workflow includes URL input, preprocessing, feature extraction, model prediction, and display of a verdict through a simple interface. The supplied presentation lists Logistic Regression, Support Vector Machine (SVM), Random Forest, and XGBoost as candidate algorithms. It presents Random Forest as the preferred model in its example. However, the accuracy and metric values in the presentation are explicitly illustrative; actual experimental results must be obtained before they can be reported as project findings.

Keywords: phishing detection, URL classification, machine learning, cybersecurity, Random Forest, feature extraction.

TABLE OF CONTENTS

Section	Page
Certificate	2
Declaration	3
Acknowledgement	4
Abstract	5
1. Introduction	7
2. Problem Statement and Objectives	8
3. Literature Background	9
4. Existing and Proposed Systems	10
5. System Requirements and Scope	11
6. System Architecture	12
7. URL Feature Extraction	13
8. Dataset and Pre-processing	14
9. Machine-Learning Algorithms	15
10. Model Training and Testing	17
11. Evaluation Metrics	18
12. Results and Discussion	19
13. Application Workflow and Interface	20
14. Security, Privacy and Ethical Considerations	21
15. Advantages and Limitations	22
16. Future Enhancements	23
17. Test Plan and Test Cases	24
18. Project Schedule and Deliverables	25
19. Conclusion	26
References	27
Appendix A: Glossary	28
Appendix B: Viva Questions	29
Appendix C: Submission Checklist	30

1. INTRODUCTION

Internet services support communication, shopping, banking, education, and entertainment. As online activity grows, cybercriminals increasingly use deceptive websites and messages to trick people into revealing private information. Phishing is one of the most common forms of online deception because it exploits trust and human attention rather than relying only on technical vulnerabilities.

1.1 Meaning of phishing

Phishing is an attempt to make a person believe that a message or website is genuine when it is controlled by an attacker. A fake sign-in page may imitate a real service and ask the visitor to enter a password. The attacker can then misuse the captured information.

1.2 Why URLs matter

A URL identifies the location of an online resource. Attackers may use look-alike spellings, misleading subdomains, long paths, unusual symbols, or shortened links to hide the destination. Examining URL characteristics can provide useful signals for automated detection.

1.3 Role of artificial intelligence

Machine-learning methods learn patterns from examples. When trained on labelled phishing and legitimate URLs, a classifier can estimate the class of a new URL based on its extracted features. The prediction is a risk indicator rather than a guarantee.

1.4 Purpose of the project

The project aims to describe a simple pipeline that accepts a URL, processes it consistently, extracts features, applies a trained model, and shows a clear result to the user.

2. PROBLEM STATEMENT AND OBJECTIVES

Phishing URLs are created and changed continuously. A blacklist-based system may not contain a URL that has only recently appeared. Hand-written rules can be bypassed by small changes in spelling or URL structure. Therefore, a data-driven detector can provide an additional layer of warning.

2.1 Problem statement

Existing approaches that depend mainly on known malicious URL lists may not recognise new phishing links immediately. This creates a need for a classifier that learns patterns from labelled examples and can be evaluated on unseen URLs.

2.2 Main objective

To describe an AI-based system that classifies URLs as potentially phishing or legitimate using URL-related features and machine-learning algorithms.

2.3 Specific objectives

• Collect labelled examples of phishing and legitimate URLs.

• Preprocess the data and extract useful features.

• Train and compare several machine-learning classifiers.

• Evaluate models using accuracy, precision, recall, F1-score, and a confusion matrix.

• Present a simple verdict through a web interface.

• Document limitations and identify improvements for future work.

2.4 Scope

The described system focuses mainly on URL characteristics. Full webpage rendering, screenshot comparison, email-body analysis, and browser-level blocking are future extensions unless they have been separately implemented.

3. LITERATURE BACKGROUND

Research into phishing detection includes blacklist matching, heuristic rules, statistical methods, and machine-learning classifiers. The supplied presentation lists several references that provide background for website classification and machine learning.

3.1 Blacklist-based detection

A blacklist stores URLs or domains already identified as malicious. It is straightforward to use and can quickly reject known threats. Its key limitation is that a new URL may not yet be present.

3.2 Feature-based machine learning

Feature-based systems represent URL or website properties as numeric values. A model learns how these values differ between labelled classes. The quality of the features and dataset strongly influences performance.

3.3 Random Forest

Breiman's work on Random Forests introduced an ensemble approach that combines decision trees. Such models can represent non-linear relationships and are commonly used as classification baselines.

3.4 Scikit-learn

Scikit-learn provides Python tools for preprocessing, model fitting, and evaluation. It supports many of the candidate classifiers described in the presentation.

3.5 Research gap and project motivation

The project presentation motivates a model that can generalise beyond a list of known URLs. The report therefore emphasises a consistent feature pipeline and testing on held-out data. The supplied presentation does not provide a full literature comparison or reproducible experimental logs, so those should be added if required by the institution.

4. EXISTING AND PROPOSED SYSTEMS

The project presentation compares existing blacklist and rule-based approaches with a proposed learning-based detector.

4.1 Existing system

Existing systems commonly check URLs against lists of known threats or apply manually written rules. They can be effective against known patterns but may miss new links and may require frequent updates. A strict rule can also incorrectly flag a legitimate website.

4.2 Proposed system

The proposed system learns from labelled examples. During prediction, the user submits a URL, the application applies preprocessing and feature extraction, and the saved classifier produces a class prediction. The interface then displays a warning or a legitimate-looking result.

4.3 Comparison

• Existing system: depends strongly on known URL lists or fixed rules.

• Proposed system: learns patterns from labelled examples.

• Existing system: newly created URLs may be absent from the blacklist.

• Proposed system: may recognise unseen URLs if their patterns resemble learned examples.

• Both approaches can make mistakes and should be evaluated.

4.4 Expected outcome

The expected outcome is a simple URL-checking workflow. The model’s real performance must be measured using a documented dataset and a held-out test set.

5. SYSTEM REQUIREMENTS AND SCOPE

The presentation suggests a Python-based implementation with common machine-learning and web-development libraries. The final report should reflect the tools actually installed and used by the team.

5.1 Software requirements

• Python 3 for the program logic.

• scikit-learn for machine-learning algorithms.

• XGBoost if the XGBoost comparison model is included.

• Pandas and NumPy for data processing.

• Flask for a basic web interface.

• Jupyter Notebook or Visual Studio Code for development.

5.2 Hardware requirements

• Intel i3 or equivalent processor or better.

• At least 4 GB RAM; 8 GB is suggested for a more comfortable workflow.

• Approximately 500 MB of free storage for code, libraries, and dataset files.

• Internet access where online datasets or host information are needed.

5.3 Functional requirements

The application should accept a URL, validate the input, extract features, load a trained model, predict a class, and display a clear result. It should handle empty or malformed input without crashing.

5.4 Non-functional requirements

The interface should be easy to use, predictions should be reasonably quick, errors should be handled clearly, and user-submitted URLs should not be unnecessarily retained.

5.5 Scope boundaries

The report describes a proposed workflow based on the presentation. It does not claim that a production-ready detector, browser extension, or live threat-intelligence integration has been deployed.

6. SYSTEM ARCHITECTURE

The architecture contains a training workflow and a prediction workflow. They share the same feature definitions so that model inputs remain consistent.

6.1 URL input

A user pastes or types a URL into the checker. The system should reject blank input and safely handle malformed strings.

6.2 Preprocessing

The input is normalised using a documented process. The system should avoid changing meaningful URL components in ways that could affect classification.

6.3 Feature extraction

The URL is transformed into a numeric feature vector. The same feature order and definitions must be used during training and prediction.

6.4 Model inference

The saved classifier receives the feature vector and returns a predicted class. If probabilities are available and calibrated, the application may show an estimated confidence.

6.5 Decision and display

The predicted class is mapped to a readable label such as “Potential phishing” or “Likely legitimate.” The interface should avoid suggesting that any automated result is infallible.

6.6 Training pipeline

Labelled data is cleaned, split, transformed, and used to fit candidate models. The best model is selected using held-out evaluation results and saved with its preprocessing steps.

7. URL FEATURE EXTRACTION

Feature extraction translates URL characteristics into data that can be processed by a classifier. The supplied presentation lists the following examples. Confirm which features are actually present in the implementation.

7.1 URL length

Long URLs can contain many path segments and parameters. Length may be useful as one signal, but many legitimate URLs are long.

7.2 Special characters

Symbols such as @ and hyphens can appear in URLs used for deception. Their presence alone does not establish that a URL is malicious.

7.3 IP address

A URL that uses a raw IP address instead of a domain name may deserve additional scrutiny, though legitimate services can also use IP addresses.

7.4 HTTPS

HTTPS encrypts the connection between a browser and a website. It does not prove that the website owner is honest; phishing websites can also use HTTPS.

7.5 Subdomains and dots

Multiple subdomain levels can make the true registrable domain harder to recognise. Correct domain parsing is important when calculating this feature.

7.6 Suspicious words

Words such as login, verify, secure, and update may appear in phishing links, but they also occur on genuine websites.

7.7 Domain age

A recently registered domain may be suspicious. Domain age normally requires a reliable source and may not be available offline.

7.8 URL shorteners

Shortened links hide the destination in their visible text. Expanding them requires care because the destination can change or redirect.

8. DATASET AND PRE-PROCESSING

The presentation references the UCI Phishing Websites dataset and lists 11,055 labelled samples, 30 features, and an 80:20 training-test split. These figures should be checked against the exact dataset file used in the implementation.

8.1 Dataset labels

The presentation describes legitimate URLs as label 0 and phishing URLs as label 1. The team should verify the label mapping in the actual dataset before training.

8.2 Cleaning

Remove duplicate records and handle missing values. Keep a record of any removed rows so that the data preparation process can be reproduced.

8.3 Encoding and scaling

Convert categorical values into the form expected by the model. Scale numeric features when required, particularly for algorithms sensitive to feature scale.

8.4 Train-test split

The presentation suggests an 80:20 split. The test set should remain separate from model fitting and model selection to provide a fair estimate of generalisation.

8.5 Data leakage

Information from the test set must not influence preprocessing choices or model fitting. If multiple URLs from the same domain are split across training and test sets, performance may appear more optimistic than performance on entirely new domains.

8.6 Data expansion

The presentation mentions PhishTank as a possible source of additional phishing URLs. Additional data should be validated, labelled, dated, and used according to the source’s terms.

9. MACHINE-LEARNING ALGORITHMS — I

The presentation lists four candidate algorithms to compare. Model choice should be based on measured results rather than on an assumed ranking.

9.1 Logistic Regression

Logistic Regression is a widely used classification baseline. It estimates class probability from a weighted combination of input features. It is usually fast and provides a useful reference point, although complex relationships may require feature engineering.

9.2 Advantages of Logistic Regression

• Fast to train on many tabular datasets.

• Relatively easy to explain.

• Can provide probability estimates.

• Useful as a baseline for comparing more complex methods.

9.3 Support Vector Machine

An SVM seeks a separating boundary between classes. Depending on the kernel and settings, it can model linear or non-linear decision boundaries. Feature scaling is often important.

9.4 Advantages and considerations of SVM

• Can perform well in high-dimensional feature spaces.

• Requires suitable kernel and parameter selection.

• Training can become expensive for large datasets.

• Probability outputs may require additional calibration.

9.5 Comparison practice

Use the same data split and evaluation metrics for both models. Record parameters and preprocessing choices so that the comparison is reproducible.

9. MACHINE-LEARNING ALGORITHMS — II

The supplied presentation identifies Random Forest as the selected model in its example and includes XGBoost for comparison. Actual selection should be supported by reproducible evaluation.

9.6 Random Forest

Random Forest combines multiple decision trees and aggregates their predictions. It can model non-linear patterns and interactions among URL features. Its performance depends on dataset quality, feature design, and parameter settings.

9.7 Random Forest strengths

It can work well on tabular features, is less dependent on feature scaling than some alternatives, and can provide feature-importance estimates. Feature importance should be interpreted carefully and does not prove causation.

9.8 XGBoost

XGBoost builds an ensemble of trees sequentially, with later trees attempting to improve on earlier errors. It includes regularisation and other controls that can help manage model complexity.

9.9 Fair model comparison

All candidate models should use the same training and test data, with model selection performed without repeatedly tuning against the test set. Compare precision and recall as well as accuracy because phishing detection errors have different consequences.

9.10 Saving the model

Save the fitted classifier together with any preprocessing object and a record of feature order, dataset version, library versions, and training settings.

10. MODEL TRAINING AND TESTING

Training means fitting a classifier to labelled examples. Testing estimates how well the fitted model works on data that was not used for fitting.

10.1 Training procedure

• Load the labelled dataset.

• Check labels, duplicates, missing values, and feature definitions.

• Split data into training and test portions.

• Fit preprocessing steps on training data only.

• Train each candidate classifier.

• Select the model using a documented validation strategy.

• Evaluate the selected model on the held-out test set.

10.2 Prediction procedure

When a new URL is entered, the application runs the same preprocessing and feature extraction steps used during training. It then sends the feature vector to the saved model and displays the predicted class.

10.3 Reproducibility

Record the dataset source and version, split method, random seed, model parameters, and software versions. This allows the team or an evaluator to repeat the experiment.

10.4 Testing beyond random split

If possible, test on URLs collected at a later time or from domains absent from training. Such evaluation better reflects changing phishing tactics than a random split alone.

10.5 Error analysis

Inspect false positives and false negatives. False positives can block legitimate sites; false negatives can allow phishing URLs to pass undetected. Both should be documented.

11. EVALUATION METRICS

A single metric does not fully describe a phishing detector. The project presentation lists accuracy, precision, recall, F1-score, and the confusion matrix.

11.1 Accuracy

Accuracy is the number of correct predictions divided by the total number of predictions. It is easy to understand but can be misleading if the classes are imbalanced.

11.2 Precision

Precision measures the proportion of URLs predicted as phishing that are actually phishing. Higher precision means fewer legitimate URLs are incorrectly flagged among the URLs labelled phishing.

11.3 Recall

Recall measures the proportion of actual phishing URLs that the detector correctly identifies. Low recall means more phishing URLs are missed.

11.4 F1-score

F1-score is the harmonic mean of precision and recall. It balances the two measures, but the appropriate operating point depends on the use case.

11.5 Confusion matrix

The matrix records true positives, true negatives, false positives, and false negatives. These counts make it easier to understand the types of mistakes made by the classifier.

11.6 Threshold selection

If the model provides probabilities, a decision threshold can be selected using validation data. The threshold should reflect the trade-off between missed phishing links and false warnings.

12. RESULTS AND DISCUSSION

The presentation contains example results, but it explicitly labels them as illustrative placeholders. They must not be described as verified experimental findings.

12.1 Accuracy figures shown in the presentation

Logistic Regression: 92.4%; SVM: 94.6%; XGBoost: 96.9%; Random Forest: 97.2%. These are illustrative values from the supplied presentation.

12.2 Random Forest metrics shown

The presentation lists illustrative precision of 97.5%, recall of 97.0%, and F1-score of 97.2%. The figures need confirmation from actual runs.

12.3 How to interpret results

If Random Forest achieves the best measured performance on a fair held-out set, the team may justify selecting it. If another model performs better or has a more suitable precision-recall balance, the final choice should reflect that evidence.

12.4 Recommended result table

For each classifier, report accuracy, precision, recall, F1-score, and confusion-matrix counts. Include the test-set size and class distribution.

12.5 Limitations of the supplied results

The presentation does not include reproducible logs, code output, or a confusion matrix. Therefore this report does not claim that the illustrative figures were measured during preparation.

Classifier	Accuracy shown	Evidence status
Logistic Regression	92.4%	Illustrative
SVM	94.6%	Illustrative
XGBoost	96.9%	Illustrative
Random Forest	97.2%	Illustrative

13. APPLICATION WORKFLOW AND INTERFACE

The presentation depicts a sample web interface named “PhishGuard | URL Checker.” The following describes the intended workflow rather than claiming a deployed application was tested.

13.1 User input

The user enters a complete URL into the input field and selects the Check action.

13.2 Validation

The application checks that the input is not blank and follows an acceptable URL format. Invalid input should produce a clear message.

13.3 Prediction

The URL is processed, features are extracted, and the trained model returns a class prediction.

13.4 Display

The interface presents a clear result, for example “Potential phishing” or “Likely legitimate.” If a confidence score is displayed, it should be explained as a model estimate rather than a guarantee.

13.5 Sample cases

The supplied presentation shows a suspicious-looking URL returning a phishing warning and a Wikipedia URL returning a legitimate label. These are sample interface examples, not verified live test outcomes.

13.6 Usability

Use readable labels, a clear warning colour, accessible text, and instructions not to enter passwords or payment information into suspicious websites.

14. SECURITY, PRIVACY AND ETHICAL CONSIDERATIONS

A phishing detector is itself a software tool and should be designed with care. Its warnings may influence user decisions, so the application should communicate uncertainty honestly.

14.1 User privacy

Avoid collecting personal data that is not needed for classification. If submitted URLs are logged for debugging, explain this to users and remove sensitive query parameters where appropriate.

14.2 Safe handling of URLs

Do not automatically open or fetch every user-submitted URL without appropriate safeguards. Server-side URL fetching can expose a system to risks such as requests to internal network addresses.

14.3 Honest warnings

A “Safe” prediction should not be represented as proof that a website is genuine. Use wording such as “No phishing pattern detected” where appropriate.

14.4 Dataset responsibility

Use datasets in accordance with their terms. Avoid publishing personal information or sensitive URLs inadvertently contained in logs.

14.5 Responsible use

The system should be used to help users avoid deceptive websites, not to create, test, or distribute phishing pages.

15. ADVANTAGES AND LIMITATIONS

The proposed approach may provide useful additional protection, but its performance depends on data and implementation choices.

15.1 Advantages

• Can learn patterns rather than depend only on known URL lists.

• Can provide rapid predictions after model loading.

• Can be retrained when new labelled examples are available.

• Can compare several classifiers under a common evaluation procedure.

• Can be integrated into a web interface.

15.2 Limitations

• Poor-quality or outdated data can reduce generalisation.

• URL-only features do not fully inspect page content or visual similarity.

• Attackers may change URL structures to evade detection.

• Host information such as domain age may require network access.

• False positives and false negatives remain possible.

• Illustrative metrics in the presentation are not proof of actual performance.

15.3 Mitigation

Use representative and updated datasets, evaluate on held-out and time-separated data, review false predictions, and communicate the uncertainty of every automated verdict.

16. FUTURE ENHANCEMENTS

The presentation proposes several directions for extending the system beyond URL-only classification.

16.1 Browser extension

A browser extension could warn users before they visit a URL. It would need efficient inference, safe update mechanisms, and a clear way to explain warnings.

16.2 Deep learning

LSTM or CNN-based approaches could be explored for raw URL text. Their benefit should be compared against simpler models using the same evaluation data.

16.3 Page content and screenshots

Additional analysis could inspect webpage text, form behaviour, and screenshots to identify visual imitation. Such features require careful handling of potentially malicious content.

16.4 Mobile and email integration

The detector could be connected to a mobile application or email workflow so users can check suspicious links before opening them.

16.5 Model monitoring

Monitor false positives and false negatives, update training data, and re-evaluate the model as phishing tactics change.

16.6 Explainability

A future version could show the signals that contributed to a warning, while avoiding overconfident explanations that imply a feature proves malicious intent.

17. TEST PLAN AND TEST CASES

Testing should verify both normal predictions and the application's handling of invalid input. The cases below are proposed test cases; they are not claimed to have been executed.

17.1 Functional tests

• Valid URL: application accepts the input and returns a prediction.

• Blank input: application displays a validation message.

• Malformed URL: application handles the input without crashing.

• Model unavailable: application reports an error rather than silently guessing.

• Repeated input: identical input produces consistent output when the model is deterministic.

17.2 Model tests

Measure metrics on a held-out test set. Review confusion-matrix counts and inspect examples of false positives and false negatives.

17.3 Security tests

Check that URL input is treated as data, not executable code. If the server fetches external URLs, test protections against internal-address access, redirect abuse, timeouts, and oversized responses.

17.4 Proposed test-case table

Document the test input category, expected behaviour, actual outcome, and pass/fail status. Do not mark tests as passed until they have been performed.

Test	Expected behaviour	Status
Blank URL	Show validation message	To be tested
Malformed URL	Handle safely	To be tested
Valid URL	Return model prediction	To be tested
Model file missing	Show clear error	To be tested

18. PROJECT SCHEDULE AND DELIVERABLES

The project can be completed in stages so that the data, model, interface, and report are developed in a controlled sequence. The schedule below is a suggested plan and should be adjusted to the actual work completed.

18.1 Suggested schedule

• Stage 1 — topic selection and background reading.

• Stage 2 — dataset collection and verification.

• Stage 3 — data cleaning and feature preparation.

• Stage 4 — model training and comparison.

• Stage 5 — evaluation and error analysis.

• Stage 6 — interface development and testing.

• Stage 7 — report preparation and presentation.

18.2 Deliverables

Potential deliverables include a cleaned dataset or data-preparation script, feature extraction code, trained model file, evaluation results, a simple URL-checking interface, and the final report.

18.3 Team responsibilities

The team may divide work among data preparation, model development, interface implementation, testing, and documentation. The actual assignment of responsibilities should be recorded by the students rather than assumed in this report.

18.4 Completion criteria

Before submission, verify that the project can be reproduced, actual results are recorded, screenshots match the implemented application, and all placeholders have been completed.

19. CONCLUSION

This report describes an AI-based phishing URL detection system intended to help identify suspicious links through machine learning.

19.1 Summary

The proposed pipeline includes labelled data, preprocessing, feature extraction, model training, prediction, and a user-facing result. Candidate algorithms include Logistic Regression, SVM, Random Forest, and XGBoost.

19.2 Main takeaway

Machine learning may identify URL patterns beyond those already recorded in a blacklist, but success depends on representative data, careful evaluation, and consistent preprocessing.

19.3 Final note

The supplied presentation’s accuracy and precision/recall values are illustrative. Actual experiments must be run before the team claims a measured performance level. A positive “Safe” result should never be treated as an absolute guarantee.

REFERENCES

1. Mohammad, R. M., McCluskey, L., and Thabtah, F. “Predicting phishing websites based on self-structuring neural network.” Neural Computing and Applications, 2014.

2. Sahingoz, O. K., et al. “Machine learning based phishing detection from URLs.” Expert Systems with Applications, 2019.

3. Breiman, L. “Random Forests.” Machine Learning, 45(1), 2001.

4. Pedregosa, F., et al. “Scikit-learn: Machine Learning in Python.” Journal of Machine Learning Research, 2011.

5. UCI Machine Learning Repository. “Phishing Websites” dataset.

6. PhishTank. Phishing URL data resource: https://phishtank.org/ .

7. Anti-Phishing Working Group (APWG). Phishing Activity Trends Reports.

These references are based on the list supplied in the project presentation. Check the exact bibliographic details and institutional citation style before submission.

APPENDIX A: GLOSSARY

Term	Meaning
AI	Artificial Intelligence; methods that perform tasks associated with intelligent behaviour.
Phishing	Deceptive activity intended to obtain sensitive information.
URL	Uniform Resource Locator; the address of an online resource.
Classifier	A model that assigns an input to a category.
Feature	A measurable input used by a machine-learning model.
Dataset	A collection of examples used for analysis or training.
Training set	Data used to fit the model.
Test set	Held-out data used to estimate model performance.
Precision	Proportion of phishing predictions that are correct.
Recall	Proportion of actual phishing examples detected.
F1-score	Harmonic mean of precision and recall.
False positive	A legitimate URL incorrectly flagged as phishing.
False negative	A phishing URL incorrectly labelled legitimate.
Random Forest	An ensemble classifier built from multiple decision trees.
SVM	Support Vector Machine; a model that separates classes using a decision boundary.
XGBoost	A gradient-boosting framework commonly used for tree-based models.

APPENDIX B: SAMPLE VIVA QUESTIONS AND ANSWERS

1. What is phishing?

Phishing is an attempt to trick users into revealing sensitive information through deceptive messages or websites.

2. Why is URL detection useful?

URL characteristics can provide signals that a link may lead to a deceptive website.

3. Why can blacklists fail?

A newly created malicious URL may not yet be listed.

4. What is feature extraction?

It converts URL properties into values that a classifier can process.

5. Name the algorithms in the project presentation.

Logistic Regression, SVM, Random Forest, and XGBoost.

6. What is a training set?

The data used to fit the model.

7. What is a test set?

Data kept separate to evaluate the model after training.

8. What is precision?

The proportion of URLs predicted as phishing that are actually phishing.

9. What is recall?

The proportion of actual phishing URLs detected by the model.

10. Does HTTPS mean a website is safe?

No. HTTPS encrypts the connection but does not prove that the website is genuine.

11. Why compare models?

To select a model based on measured performance and the project's requirements.

12. What is a false positive?

A legitimate URL incorrectly flagged as phishing.

13. What is a false negative?

A phishing URL incorrectly labelled legitimate.

14. What is Random Forest?

An ensemble method that combines multiple decision trees.

15. Are the accuracy figures verified?

Not in the supplied presentation; they are marked as illustrative and need actual experimental confirmation.

APPENDIX C: FINAL SUBMISSION CHECKLIST

• Fill in the college/university name, department, academic year, and registration numbers if required.

• Confirm the spelling of all student and guide names.

• Verify the dataset version, sample count, feature count, and label mapping.

• Run the model experiments and replace illustrative metrics with actual results.

• Include the real confusion matrix and screenshots if required.

• Ensure the feature extraction used in the application matches the training pipeline.

• Test blank input, malformed input, valid URLs, and missing model files.

• Check references and apply the institution’s required citation format.

• Obtain the guide’s review and signatures where required.

• Remove all uncompleted placeholders before final submission.

Prepared for academic review by:

Daniel Raj V • Karthikeyan R • Justin Imre J

Project Guide: **Ms. Divya Bairavi**

Project title: **AI-Based Phishing URL Detection System**